Memento

Memento · Save/restore text editor snapshots · Build with Waqar

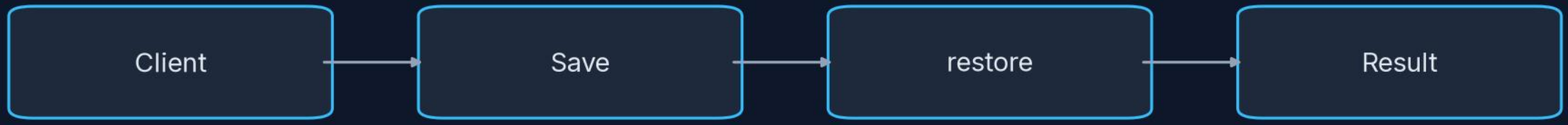
Today's Hook

- Problem: Save/restore text editor snapshots
- Pattern: Memento — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Save/restore text editor snapshots
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Memento: Save/restore text editor snapshots

Real-World Scenario

- Running example: Save/restore text editor snapshots
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.memento;

import com.buildwithwaqar.patterns.gof.behavioral.memento.TextEditor;
/** Problem: save and restore document checkpoints. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Memento problem: editor checkpoints ===");
    var ed = new TextEditor(); var h = new TextEditor.History();
    ed.type("v1"); h.push(ed.save());
    ed.type("-v2"); System.out.println("now=" + ed.text());
    ed.restore(h.pop()); System.out.println("restored=" + ed.text());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.memento;

import java.util.ArrayDeque;
import java.util.Deque;

public final class TextEditor {
    private String text = "";

    public void type(String chunk) { text += chunk; }
    public String text() { return text; }

    public Memento save() { return new Memento(text); }
    public void restore(Memento m) { text = m.state(); }

    public record Memento(String state) {}

    /** Caretaker */
    public static final class History {
        private final Deque<Memento> stack = new ArrayDeque<>();
        public void push(Memento m) { stack.push(m); }
        public Memento pop() { return stack.pop(); }
        public boolean isEmpty() { return stack.isEmpty(); }
    }
```

Solution (2)

```
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.memento;

import com.buildwithwaqar.patterns.gof.behavioral.memento.TextEditor;
/** Problem: save and restore document checkpoints. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Memento problem: editor checkpoints ===");
    var ed = new TextEditor(); var h = new TextEditor.History();
    ed.type("v1"); h.push(ed.save());
    ed.type("-v2"); System.out.println("now=" + ed.text());
    ed.restore(h.pop()); System.out.println("restored=" + ed.text());
  }
}
```

Walkthrough Output

```
=== Memento problem: editor checkpoints ===
now=v1-v2
restored=v1
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Memento.

Related Patterns

- Contrast Memento with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Memento → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.